

Axiomatic and Formal Description of the CatBoost + CNN Monthly Return Pipeline

Abstract

We study a supervised learning problem on a monthly company-level panel. The objective is to predict the monthly gross return of a company using a two-stage model: a tabular first-stage learner based on CatBoost and a temporal residual learner based on a one-dimensional convolutional neural network. The pipeline is leakage-safe, chronological, and split-aware. This document states the problem axiomatically, defines the preprocessing operators mathematically, maps the code to formal transformations, presents pseudocode, and then develops CatBoost and CNN from first principles in appendices.

Contents

1	Problem Statement	3
2	Axiomatic Framework	3
2.1	Axiom 1: Panel Identity	3
2.2	Axiom 2: Chronological Order	3
2.3	Axiom 3: No Future Leakage	3
2.4	Axiom 4: Split Integrity	4
2.5	Axiom 5: Deterministic Canonicalization	4
3	Data Model and Notation	4
4	Approach Overview	4
4.1	Stage 1: CatBoost	4
4.2	Stage 2: CNN on Residuals	5
4.3	Final Prediction	5
5	Code-to-Mathematics Correspondence	5
5.1	Reading a Split	5
5.2	Dropping Previously Engineered Features	5
5.3	Type Coercion	5
5.4	Duplicate Collapse	6
5.5	Lag Feature Construction	6
5.6	Rolling Feature Construction	6
5.7	Month Index	7
5.8	Missingness Indicators	7
5.9	Feature Set Filter	7
5.10	Chronological Split Verification	7
5.11	CatBoost Feature Frame	7

5.12	CatBoost Pool and Training	7
5.13	Residual Construction	8
5.14	CNN Sequence Split	8
5.15	Encoding for the CNN	8
5.16	Standardization for the CNN	8
5.17	Sequence Construction	8
5.18	Residual CNN Training	8
6	Training Objectives and Evaluation	9
6.1	CatBoost Objective	9
6.2	CNN Objective	9
6.3	Regression Metrics	9
6.4	Directional Accuracy	9
6.5	Monthly Information Coefficient	9
6.6	Top-K Return Metric	9
6.7	Residual Diagnostics	10
7	Pseudocode	10
A	CatBoost from First Principles	10
A.1	Supervised Regression as Risk Minimization	11
A.2	Function Space and Additive Modeling	11
A.3	Gradient Boosting Step	11
A.4	Decision Trees as Partition Functions	11
A.5	Why Trees Work Well Here	12
A.6	Squared Loss and Leaf Values	12
A.7	Categorical Variables	12
A.8	Why Ordered Statistics Matter	12
A.9	Optimization Objective in the Notebook	12
B	CNN from First Principles	12
B.1	Discrete Sequence Data	13
B.2	One-Dimensional Convolution	13
B.3	Multiple Filters and Feature Maps	13
B.4	Nonlinearity	13
B.5	Pooling	13
B.6	Linear Readout	13
B.7	Why Convolution Fits the Problem	14
B.8	Backpropagation and Training	14
B.9	Input Channels in the Notebook	14
C	Recursive Definitions	14
C.1	Recursive Definition of a Company History	14
C.2	Recursive Definition of Lagged Features	14
C.3	Recursive Definition of the Final Prediction	15
D	Interpretation	15
E	Conclusion	15

1 Problem Statement

We consider a panel of monthly company observations. Each observation belongs to exactly one company and exactly one month. The prediction target is the monthly gross return. The goal is to construct a function that maps the information available at month t for company j to an estimate of the realized return at that month.

Let $j \in \{1, \dots, J\}$ index companies and $t \in \{1, \dots, T_j\}$ index months available for company j . Let the realized return be

$$y_{j,t} \in \mathbb{R}.$$

Let the raw observation vector be denoted by $r_{j,t}$ and let the engineered feature vector be denoted by $x_{j,t}$. The learning objective is to approximate the conditional relationship

$$y_{j,t} \approx f(x_{j,t}),$$

where f is not a single function in this pipeline, but a composition

$$f(x_{j,t}) = f_{\text{CB}}(x_{j,t}) + f_{\text{CNN}}(\mathcal{S}_{j,t}).$$

The first term captures structured tabular and cross-sectional signal. The second term captures remaining temporal structure in the residuals.

2 Axiomatic Framework

We state the pipeline under a small axiom system.

2.1 Axiom 1: Panel Identity

Every row corresponds to one and only one pair (j, t) after duplicate collapse. Formally, there exists an injective mapping from the processed table rows to company-month keys.

2.2 Axiom 2: Chronological Order

For each fixed company j , the months are totally ordered by time:

$$t_1 < t_2 \implies \text{month}(t_1) \prec \text{month}(t_2).$$

All lagged and rolling constructions must respect this order.

2.3 Axiom 3: No Future Leakage

Every feature at time t must be a deterministic function of data available at or before t . If ϕ denotes the feature construction map, then

$$x_{j,t} = \phi(\{r_{j,\tau} : \tau \leq t\})$$

and never depends on $r_{j,\tau}$ for $\tau > t$.

2.4 Axiom 4: Split Integrity

The train, validation, and test splits are disjoint in time. If $\mathcal{T}_{\text{train}}, \mathcal{T}_{\text{val}}, \mathcal{T}_{\text{test}}$ denote the month sets, then

$$\mathcal{T}_{\text{train}} \cap \mathcal{T}_{\text{val}} = \emptyset, \quad \mathcal{T}_{\text{val}} \cap \mathcal{T}_{\text{test}} = \emptyset.$$

In the implemented pipeline, train ends before validation begins, and validation ends before test begins.

2.5 Axiom 5: Deterministic Canonicalization

Whenever duplicate or malformed observations are present, the preprocessing stage reduces them to a unique canonical representation using deterministic rules. This ensures repeatability and stable feature construction.

3 Data Model and Notation

We define the raw panel as

$$\mathcal{D}^{\text{raw}} = \{(j, t, r_{j,t}, m_{j,t}, c_{j,t})\},$$

where:

- $r_{j,t}$ denotes the target-return-bearing row data,
- $m_{j,t}$ denotes numeric and mixed-type metadata,
- $c_{j,t}$ denotes categorical labels.

The processed panel is denoted by

$$\mathcal{D} = \{(j, t, x_{j,t}, y_{j,t})\}.$$

The final feature map is a deterministic transformation

$$\phi : \mathcal{D}^{\text{raw}} \rightarrow \mathbb{R}^d.$$

The sequence map is

$$\psi : \mathcal{D} \rightarrow \mathbb{R}^{d \times L},$$

where L is the sequence length used by the CNN.

4 Approach Overview

We use a two-stage residual learning strategy.

4.1 Stage 1: CatBoost

CatBoost learns a function $f_{\text{CB}} : \mathbb{R}^d \rightarrow \mathbb{R}$ that predicts the target from tabular features. It captures nonlinear cross-sectional dependencies, missingness structure, and categorical interactions.

4.2 Stage 2: CNN on Residuals

The second model learns a correction term from a causal sequence of past features. If the first-stage prediction is $\hat{y}_{j,t}^{\text{CB}}$, then the residual is

$$e_{j,t} = y_{j,t} - \hat{y}_{j,t}^{\text{CB}}.$$

The CNN learns a mapping from a sequence tensor to this residual.

4.3 Final Prediction

The combined predictor is

$$\hat{y}_{j,t} = \hat{y}_{j,t}^{\text{CB}} + \hat{e}_{j,t}^{\text{CNN}}.$$

This decomposition is the central modeling hypothesis of the notebook.

5 Code-to-Mathematics Correspondence

This section formalizes the major code blocks in the notebook. The objective is not merely to narrate what the code does, but to state the exact mathematical operator induced by each block.

5.1 Reading a Split

The function `read_split(path, split_name)` reads a CSV file and appends a split label.

Mathematically, if $\mathcal{D}_s$ is the set of rows in split $s \in \{\text{train, validation, test}\}$, then

$$\mathcal{D}_s = \{(u, s) : u \in \text{CSV}(s)\}.$$

The split label is an exogenous tag used for bookkeeping and later filtering.

5.2 Dropping Previously Engineered Features

The function `drop_previous_engineering` removes stale engineered columns if they already exist. Let C be the full column set and let C_{drop} be the set of columns to remove. Then the operator is the projection

$$\pi_{C \setminus C_{\text{drop}}}(\mathcal{D}).$$

This projection prevents feature duplication and prevents recursive application of the same engineering rule from stacking derived variables on top of derived variables.

5.3 Type Coercion

The function `coerce_panel_types` performs three canonical transformations.

Date coercion. The month field is mapped to a time element in a totally ordered time set. Let $\tau : \text{strings} \rightarrow \mathcal{T}$ be the conversion into timestamps.

Identifier coercion. The company identifier is mapped into a nullable integer space. This does not change the entity identity; it only standardizes representation.

Target coercion. The return column is mapped into $\mathbb{R}$.

For every column q not in the exempted set $E = \{\text{co_code}, \text{Month}, \text{monthly_gross_return}, \text{source_split}\} \cup C_{\text{cat}}$, we apply

$$q \mapsto \nu(q),$$

where ν is numeric coercion with missing-value propagation.

5.4 Duplicate Collapse

The function `collapse_duplicate_company_months` enforces the uniqueness of the key (j, t) .

Define an equivalence relation on rows by

$$u \sim v \iff (j_u, t_u) = (j_v, t_v).$$

Each equivalence class corresponds to one company-month key. The code selects the last representative of each class. If $[u]$ is an equivalence class ordered by appearance, then the canonical representative is

$$\rho([u]) = \max_{\prec}[u],$$

where $\prec$ is the input ordering. The resulting processed panel satisfies key uniqueness.

5.5 Lag Feature Construction

The function `engineer_features` constructs lagged returns. Let $y_{j,t}$ be the target return. For lag $\ell \in \{1, 3, 6\}$,

$$\text{lag_ret}_\ell(j, t) = y_{j, t-\ell}.$$

This is a shift operator on the time index.

If `log_ret` exists, then for the same lags,

$$\text{log_ret_lag}_\ell(j, t) = \text{log_ret}(j, t - \ell).$$

These are causal by construction because they only use earlier months.

5.6 Rolling Feature Construction

Let $\bar{y}_{j,t}^{(w)}$ denote the rolling mean with window $w \in \{3, 6\}$. The code first shifts the target by one month and then computes a rolling statistic on the shifted series. Hence

$$\bar{y}_{j,t}^{(w)} = \frac{1}{|I_{j,t}^{(w)}|} \sum_{\tau \in I_{j,t}^{(w)}} y_{j,\tau},$$

where $I_{j,t}^{(w)}$ is the set of the previous w observations ending at $t - 1$, truncated when fewer than w historical values exist.

Similarly, the rolling standard deviation is

$$s_{j,t}^{(w)} = \sqrt{\frac{1}{|I_{j,t}^{(w)}| - 1} \sum_{\tau \in I_{j,t}^{(w)}} (y_{j,\tau} - \bar{y}_{j,t}^{(w)})^2}$$

when at least two observations exist; otherwise the sample variance semantics inherited from the implementation apply.

5.7 Month Index

The month index is a deterministic calendar embedding:

$$\text{month_index}(j, t) = 12 \cdot \text{year}(j, t) + \text{month}(j, t).$$

This is a monotone index on the calendar time axis.

5.8 Missingness Indicators

For each feature column f , the indicator is

$$\text{is_missing}_f(j, t) = \begin{cases} 1, & \text{if } f(j, t) \text{ is missing,} \\ 0, & \text{otherwise.} \end{cases}$$

This is a binary feature measuring the existence of information, not its magnitude.

5.9 Feature Set Filter

The feature set excludes identifiers, the target, split labels, and internal outputs. Let C be the total column set and let E be the excluded set. Then the model feature set is

$$C_{\text{feat}} = C \setminus E.$$

The categorical subset is

$$C_{\text{cat}}^{\text{feat}} = C_{\text{feat}} \cap C_{\text{cat}}.$$

5.10 Chronological Split Verification

Let $\max \mathcal{T}_{\text{train}}$ denote the latest train month, $\min \mathcal{T}_{\text{val}}$ the earliest validation month, and $\min \mathcal{T}_{\text{test}}$ the earliest test month. The code enforces

$$\max \mathcal{T}_{\text{train}} < \min \mathcal{T}_{\text{val}} < \min \mathcal{T}_{\text{test}}.$$

This guarantees that the training process sees only past data relative to validation and test.

5.11 CatBoost Feature Frame

The function `make_feature_frame` reindexes rows to C_{feat} and normalizes categorical entries as strings. This is a canonicalization map

$$\chi : \mathcal{D} \rightarrow \mathbb{R}^d \times \Sigma^{|C_{\text{cat}}^{\text{feat}}|},$$

where Σ is the categorical alphabet.

5.12 CatBoost Pool and Training

The function `make_pool` constructs the supervised learning pair

$$(X, y) = (\mathbf{X}_{\text{train}}, \mathbf{y}_{\text{train}})$$

with categorical feature indices specified explicitly.

Training solves an empirical risk minimization problem of the form

$$\hat{f}_{\text{CB}} = \operatorname{argmin}_{f \in \mathcal{F}_{\text{CB}}} \sum_{(x_i, y_i) \in \mathbf{X}_{\text{train}}} \ell(y_i, f(x_i)),$$

with $\ell(y, \hat{y}) = (y - \hat{y})^2$ in the implementation.

5.13 Residual Construction

For each non-training row, the base residual is

$$r_{j,t}^{\text{base}} = y_{j,t} - \hat{y}_{j,t}^{\text{CB}}.$$

The training split assigns no residual supervision to the CNN stage. Validation and test residuals are the OOS targets.

5.14 CNN Sequence Split

The validation split is further partitioned into a CNN-train subset and a CNN-holdout subset. Let $\mathcal{T}_{\text{val}} = \mathcal{T}_{\text{cnn_train}} \cup \mathcal{T}_{\text{cnn_holdout}}$ with disjoint union. This creates a second-level chronological separation.

5.15 Encoding for the CNN

The function `encode_for_cnn` maps categorical columns to integer codes and numeric columns to floating-point values. Formally, for each column f :

$$\eta_f(x) = \begin{cases} \text{code}(x), & f \in C_{\text{cat}}^{\text{feat}}, \\ \text{to_float}(x), & f \notin C_{\text{cat}}^{\text{feat}}. \end{cases}$$

The output is an encoded feature matrix used only for sequence modeling.

5.16 Standardization for the CNN

The sequence features are standardized with empirical mean and standard deviation computed from the designated sequence-training rows. If z denotes a raw encoded feature dimension and μ, σ denote its training mean and standard deviation, then

$$\tilde{z} = \frac{z - \mu}{\sigma}.$$

If σ is numerically degenerate, it is replaced by 1.

5.17 Sequence Construction

For each company j and each eligible month t , the sequence tensor is

$$\mathcal{S}_{j,t} = \begin{bmatrix} \tilde{x}_{j,t-L+1} \\ \tilde{x}_{j,t-L+2} \\ \vdots \\ \tilde{x}_{j,t} \end{bmatrix}^{\top} \in \mathbb{R}^{d \times L},$$

where $L = \text{SEQ_LEN}$. The transpose produces channel-first input for the CNN.

5.18 Residual CNN Training

The sequence model solves

$$\hat{g} = \underset{g \in \mathcal{F}_{\text{CNN}}}{\text{argmin}} \sum_{(\mathcal{S}_i, r_i) \in \mathcal{S}_{\text{train}}} (r_i - g(\mathcal{S}_i))^2.$$

The final predictor is

$$\hat{y}_i = \hat{y}_i^{\text{CB}} + \hat{g}(\mathcal{S}_i).$$

6 Training Objectives and Evaluation

6.1 CatBoost Objective

CatBoost is fit with RMSE loss. The empirical objective is

$$\mathcal{L}_{\text{CB}}(f) = \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i))^2.$$

6.2 CNN Objective

The CNN is fit with mean squared error on residuals:

$$\mathcal{L}_{\text{CNN}}(g) = \frac{1}{m} \sum_{i=1}^m (r_i - g(\mathcal{S}_i))^2.$$

6.3 Regression Metrics

For any prediction vector $\hat{y}$ and truth vector y over a finite evaluation set:

$$\text{RMSE}(y, \hat{y}) = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2},$$

$$\text{MAE}(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|.$$

The coefficient of determination is

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}, \quad \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i.$$

6.4 Directional Accuracy

Directional accuracy is the proportion of sign agreement:

$$\text{DA}(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{\text{sign}(y_i) = \text{sign}(\hat{y}_i)\}.$$

6.5 Monthly Information Coefficient

For each month t , the code computes the rank correlation between true and predicted returns. If $\text{rank}(\cdot)$ denotes within-month ranking and ρ denotes Pearson correlation, then

$$\text{IC}(t) = \rho(\text{rank}(y_{\cdot,t}), \text{rank}(\hat{y}_{\cdot,t})).$$

6.6 Top-K Return Metric

For each month, the top K predictions are selected and their realized returns averaged. If $\mathcal{I}_{t,K}$ denotes the index set of the K largest predicted returns at time t , then

$$\text{TopK}(t) = \frac{1}{K} \sum_{i \in \mathcal{I}_{t,K}} y_{i,t}.$$

6.7 Residual Diagnostics

Let r^{base} denote the CatBoost residual and r^{final} the final residual after CNN correction. Variance reduction is

$$\text{VR} = 1 - \frac{\text{Var}(r^{\text{final}})}{\text{Var}(r^{\text{base}})}.$$

The one-step company-level autocorrelation is computed by grouping by company and using lag one within each group.

7 Pseudocode

The following pseudocode appears before the appendices and abstracts the operational logic of the notebook.

INPUT: train.csv, validation.csv, test.csv

1. Read each split and attach a split label.
2. Concatenate the three splits temporarily.
3. Remove stale engineered columns.
4. Convert Month to datetime, co_code to integer, target to numeric.
5. Normalize categorical labels as strings and fill missing categories.
6. Sort by (co_code, Month).
7. Collapse duplicate (co_code, Month) rows by keeping the last row.
8. For each company:
 - a. compute lag returns for 1, 3, and 6 months,
 - b. compute rolling mean and rolling standard deviation,
 - c. compute lagged log-return values when available,
 - d. compute month_index,
 - e. add missingness indicators.
9. Split the processed table back into train, validation, and test.
10. Build CatBoost feature matrices and fit CatBoost on train.
11. Predict on validation and test; compute base residuals.
12. Partition validation into CNN-train and CNN-holdout months.
13. Encode sequence features and standardize them on training statistics.
14. Build causal windows of length SEQ_LEN for eligible rows.
15. Fit the CNN on residual sequences.
16. Predict residual corrections for validation and test sequences.
17. Form final predictions as CatBoost prediction + CNN correction.
18. Compute RMSE, MAE, R^2 , IC, directional accuracy, and diagnostics.

A CatBoost from First Principles

This appendix develops CatBoost-style gradient boosting from the beginning. The notation is intentionally explicit.

A.1 Supervised Regression as Risk Minimization

Let $\{(x_i, y_i)\}_{i=1}^n$ be a training set with $x_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$. The regression problem is to find a function f that minimizes the expected risk

$$\mathcal{R}(f) = \mathbb{E}[\ell(Y, f(X))].$$

Because the data distribution is unknown, we minimize the empirical risk

$$\hat{\mathcal{R}}(f) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, f(x_i)).$$

For squared loss, $\ell(y, f(x)) = (y - f(x))^2$.

A.2 Function Space and Additive Modeling

CatBoost builds an additive model

$$F_M(x) = \sum_{m=0}^M \eta_m h_m(x),$$

where each h_m is a decision tree and η_m is a step size. The model starts from a constant baseline

$$F_0(x) = \underset{c}{\operatorname{argmin}} \sum_{i=1}^n (y_i - c)^2 = \bar{y}.$$

A.3 Gradient Boosting Step

At boosting iteration m , the model has current prediction F_{m-1} . The negative gradient of the squared loss is

$$g_i^{(m)} = - \left. \frac{\partial}{\partial F} \ell(y_i, F) \right|_{F=F_{m-1}(x_i)} = y_i - F_{m-1}(x_i).$$

Thus the negative gradient is exactly the residual.

The next tree h_m is chosen to approximate these residuals:

$$h_m \approx \underset{h \in \mathcal{H}}{\operatorname{argmin}} \sum_{i=1}^n (g_i^{(m)} - h(x_i))^2.$$

Then

$$F_m(x) = F_{m-1}(x) + \eta_m h_m(x).$$

A.4 Decision Trees as Partition Functions

A regression tree partitions feature space into leaves $\{R_1, \dots, R_K\}$. On each leaf the tree is constant:

$$h(x) = \sum_{k=1}^K c_k \mathbf{1}\{x \in R_k\}.$$

The parameters are the regions R_k and the leaf values c_k .

A.5 Why Trees Work Well Here

The panel contains nonlinear interactions, threshold effects, and missingness structure. Trees naturally represent these via axis-aligned splits. A split such as

$$x_j \leq \tau$$

divides observations into two groups, allowing separate leaf predictions.

A.6 Squared Loss and Leaf Values

If the tree structure is fixed, the optimal leaf value in region R_k under squared loss is the mean residual on that leaf:

$$c_k^* = \frac{1}{|I_k|} \sum_{i \in I_k} g_i^{(m)},$$

where $I_k = \{i : x_i \in R_k\}$.

A.7 Categorical Variables

CatBoost is designed to handle categorical features without requiring manual one-hot encoding. At an abstract level, a categorical variable $c \in \mathcal{C}$ is mapped into a representation usable by tree splits, often through target-statistic-based encodings or ordered statistics that reduce leakage. The essential mathematical point is that a categorical feature is not treated as an ordered real number unless the encoding makes that order explicit.

The notebook supplies categorical columns directly to CatBoost. Therefore the model internally performs the category handling required to optimize split quality while preserving leakage control.

A.8 Why Ordered Statistics Matter

If a category encoding uses target information, then naive averaging can leak the label into the feature. CatBoost addresses this by using permutations or ordered schemes so that the encoding for a row is computed from earlier rows only. This is consistent with the pipeline’s leakage-free principle.

A.9 Optimization Objective in the Notebook

The notebook uses RMSE as the evaluation criterion and trains the model with the CatBoost regression objective. In formal terms, the fitted model approximates

$$\hat{f}_{\text{CB}} = \operatorname{argmin}_{f \in \mathcal{F}_{\text{tree}}} \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i))^2.$$

The GPU/CPU choice changes implementation, not the mathematical objective.

B CNN from First Principles

This appendix develops the convolutional residual learner from the beginning.

B.1 Discrete Sequence Data

The CNN consumes a tensor

$$S_i \in \mathbb{R}^{d \times L},$$

where d is the number of channels and L is the sequence length. Each column corresponds to one time step, and each row corresponds to one feature channel.

B.2 One-Dimensional Convolution

Let $W \in \mathbb{R}^{d \times k}$ be a filter of width k and let $b \in \mathbb{R}$ be a bias term. For a valid temporal index τ , the pre-activation at location τ is

$$z_\tau = b + \sum_{c=1}^d \sum_{u=0}^{k-1} W_{c,u} S_{c,\tau+u}.$$

This is the discrete convolution operator specialized to one-dimensional time.

B.3 Multiple Filters and Feature Maps

If there are K filters, then the layer output is a family of feature maps

$$z^{(1)}, \dots, z^{(K)}.$$

Each filter detects a different temporal pattern. One filter may respond to momentum-like patterns, another to reversals, another to volatility bursts.

B.4 Nonlinearity

After convolution, the model applies ReLU:

$$\sigma(u) = \max(0, u).$$

Thus the activated feature map is

$$a_\tau = \sigma(z_\tau).$$

The nonlinearity allows the CNN to represent non-affine functions of the input sequence.

B.5 Pooling

The notebook uses adaptive average pooling to reduce each channel to a single scalar. If $a^{(k)}$ is the activated output of filter k , then the pooled representation is

$$p_k = \frac{1}{|\Omega_k|} \sum_{\tau \in \Omega_k} a_\tau^{(k)},$$

where Ω_k is the set of positions retained by the pooling operator.

B.6 Linear Readout

The pooled vector $p \in \mathbb{R}^K$ is passed through a linear map

$$\hat{r} = w^\top p + b.$$

This is the residual prediction.

B.7 Why Convolution Fits the Problem

The residual correction should depend on local time patterns rather than global rearrangements. Convolution is appropriate because it is translation-equivariant along time: the same temporal pattern can be recognized regardless of its location within the window.

B.8 Backpropagation and Training

The CNN parameters are trained by minimizing MSE on residuals:

$$\hat{\mathcal{L}}_{\text{CNN}} = \frac{1}{m} \sum_{i=1}^m (r_i - \hat{r}_i)^2.$$

Gradient descent updates parameters in the direction of decreasing loss. If θ denotes all CNN parameters, then the generic update is

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} \hat{\mathcal{L}}_{\text{CNN}},$$

with learning rate $\alpha > 0$.

B.9 Input Channels in the Notebook

The notebook augments the sequence channels with the CatBoost prediction. Mathematically, if $x_{j,t}$ is the base engineered vector, then the CNN input at time t includes

$$\tilde{x}_{j,t} = [x_{j,t}; \hat{y}_{j,t}^{\text{CB}}].$$

This allows the CNN to learn a direct correction to the first-stage forecast.

C Recursive Definitions

We define the core objects recursively so that every higher-level object is built from lower-level objects.

C.1 Recursive Definition of a Company History

For a fixed company j , define the history up to time t recursively by

$$H_{j,1} = r_{j,1},$$

and for $t \geq 2$,

$$H_{j,t} = (H_{j,t-1}, r_{j,t}).$$

This history object is the minimal causal input for feature construction.

C.2 Recursive Definition of Lagged Features

Let $L_{\ell}(y)$ denote the lag- ℓ operator. Then

$$L_1(y)_{j,t} = y_{j,t-1},$$

and recursively,

$$L_{\ell+1}(y)_{j,t} = L_{\ell}(y)_{j,t-1}.$$

This makes each lag a repeated application of the same primitive shift.

C.3 Recursive Definition of the Final Prediction

The final prediction is built from the base learner and the residual learner:

$$\begin{aligned}\hat{y}_{j,t}^{(0)} &= \hat{y}_{j,t}^{\text{CB}}, \\ \hat{y}_{j,t}^{(1)} &= \hat{y}_{j,t}^{(0)} + \hat{e}_{j,t}^{\text{CNN}}.\end{aligned}$$

This is the final output of the pipeline.

D Interpretation

The notebook implements a structured decomposition of the target into a tabular component and a temporal correction. CatBoost addresses heterogeneous categorical and numerical signals in the cross-section. The CNN addresses serial dependence in the residuals. The preprocessing pipeline is the mechanism that makes this decomposition mathematically valid by preserving chronology, generating causal features, and enforcing unique company-month keys.

E Conclusion

We have defined the problem, the axioms, the preprocessing operators, the model decomposition, and the training and evaluation criteria in a formal notation. The appendices derived the CatBoost and CNN components from first principles so that the entire notebook can be read as a sequence of mathematical transformations rather than as a collection of ad hoc code blocks.

F Notation Table

Symbol	Meaning	Type / Set	Notes	Physical Interpretation
j	Company index	Integer	Identifies a firm in the panel	A specific company (e.g., <code>co_code</code>) in the dataset
t	Month index	Integer / time	Chronological index	A specific calendar month (e.g., "2020-01-01") for a firm
J	Number of companies	Positive integer	Total cross-sectional size	Total number of unique companies/stocks in the dataset
T_j	Number of observed months for company j	Positive integer	Can vary across firms	Number of months with data available for this specific company
$y_{j,t}$	Observed monthly gross return	Real number	Prediction target	The actual return achieved by the stock in month t
$r_{j,t}$	Raw observation record	Data object	Pre-engineering row-level data	A single row for company j at time t from the raw CSV
$x_{j,t}$	Engineered tabular feature vector	$\mathbb{R}^d$	CatBoost input	A row from the dataset containing all encoded features

Symbol	Meaning	Type / Set	Notes	Physical Interpretation
$S_{j,t}$	Sequence tensor	$\mathbb{R}^{d \times L}$	CNN input built from past features	A sequence of the past L months of feature rows for a company
L	Sequence length	Positive integer	Window size used by CNN	The number of historical months in the CNN window
d	Number of features / channels	Positive integer	Feature dimension after encoding	Total number of input variables/columns per row
f	Full predictor	Function	Combined model	The entire ML pipeline predicting stock returns
f_{CB}	CatBoost predictor	Function	First-stage tabular learner	The initial tree model predicting from a flat feature vector
f_{CNN}	CNN residual predictor	Function	Second-stage temporal learner	The 1D CNN predicting target corrections
$\hat{y}_{j,t}$	Final prediction	Real number	CatBoost plus CNN correction	The pipeline's ultimate guess of the firm's return
$\hat{y}_{j,t}^{\text{CB}}$	CatBoost prediction	Real number	First-stage output	CatBoost's initial guess of the return
$\hat{e}_{j,t}^{\text{CNN}}$	CNN residual prediction	Real number	Second-stage output	The CNN's predicted correction to CatBoost's error
$e_{j,t}$	Residual	Real number	$y_{j,t} - \hat{y}_{j,t}^{\text{CB}}$	The actual error CatBoost made against the real return
$\mathcal{D}^{\text{raw}}$	Raw dataset	Set of records	Preprocessing input	The unprocessed CSV datasets (<code>train.csv</code> , etc.)
$\mathcal{D}$	Processed dataset	Set of records	Feature-engineered panel	The fully engineered DataFrame containing all features
$\mathcal{D}_s$	Split s dataset	Set of records	$s \in \{\text{train, validation, test}\}$	A specific subset of the data like the training set
ϕ	Feature construction map	Function	Uses only current and past information	The code logic deriving features from raw data
ψ	Sequence construction map	Function	Maps panel rows to sequence tensors	The logic stacking sequential rows into PyTorch tensors
$\mathcal{T}$	Time domain	Ordered set	Calendar time support	The set of all calendar months represented in the panel
$\mathcal{F}_{\text{CB}}$	CatBoost hypothesis class	Function class	Set of tabular predictors	All possible tree ensembles CatBoost could learn
$\mathcal{F}_{\text{CNN}}$	CNN hypothesis class	Function class	Set of sequence predictors	All possible weight configurations for the CNN

Symbol	Meaning	Type / Set	Notes	Physical Interpretation
$\mathcal{L}_{\text{CB}}$	CatBoost loss	Scalar objective	Squared error objective	The Root Mean Squared Error (RMSE) minimized by CatBoost
$\mathcal{L}_{\text{CNN}}$	CNN loss	Scalar objective	MSE on residuals	The Mean Squared Error (MSE) minimized by the CNN
$\hat{\mathcal{R}}(f)$	Empirical risk	Scalar objective	Average training loss	The average error calculated over the actual training rows
$\mathbb{R}$	Real numbers	Set	Continuous values	Continuous numerical values
$\mathbb{N}$	Natural numbers	Set	Positive integers	Integer counts
$\mathbf{1}$	Indicator function	Map	$\mathbf{1}\{\cdot\}$ equals 1 when condition holds	A binary/boolean flag (e.g., missing value indicators)
$\text{Var}(\cdot)$	Variance	Scalar statistic	Used in residual diagnostics	Statistical spread of the relative return errors
$\mathbb{E}[\cdot]$	Expectation	Operator	Used in risk formulation	The expected value or population average
α	Learning rate	Positive real number	CNN optimization step size	Step size for the optimizer updating CNN weights
w	Linear readout weights	Vector	Final CNN head parameters	The weights of the final linear layer in the CNN
b	Linear readout bias	Scalar	Final CNN head intercept	The intercept of the final linear layer in the CNN
W	Convolution kernel weights	Tensor	CNN filter parameters	Filter weights finding temporal patterns
z_τ	Convolution pre-activation	Real number	Output before nonlinearity	The raw CNN unit output before applying the ReLU activation
a_τ	Activation value	Real number	Output after ReLU	The CNN unit output after applying ReLU
p_k	Pooled feature value	Real number	Adaptive average pooling output	The metric summarizing a CNN temporal feature map
R_k	Tree leaf region	Subset of feature space	CatBoost/tree partition cell	A region defined by tree splits grouping similar firms
c_k	Leaf value	Real number	Constant prediction on a leaf	The predicted return assigned to all firms in leaf R_k
$g_i^{(m)}$	Negative gradient / residual at boosting step m	Real number	CatBoost boosting target	Unexplained return that the next tree attempts to predict
F_m	Boosted ensemble after step m	Function	Additive tree ensemble	The combined return prediction of the first m trees

Symbol	Meaning	Type / Set	Notes	Physical Interpretation
$H_{j,t}$	Company history object	Sequence object	Recursive history representation	All known data for a company up to a given month
lag_ret_ℓ	Lagged return feature	Real number	ℓ -month lag	The actual gross return of the stock ℓ months prior
log_ret_lag_ℓ	Lagged log-return feature	Real number	Lagged log-return	The actual log-return of the stock ℓ months prior
rolling_mean_w	Rolling mean feature	Real number	Window size w	The average return of the stock over the last w months
rolling_std_w	Rolling standard deviation feature	Real number	Window size w	Volatility of the stock's returns over the last w months
month_index	Calendar index	Integer	Year-month encoding	Integer indicating absolute calendar time (e.g., $2020 \times 12 + 1$)
is_missing_f	Missingness indicator	$\{0, 1\}$	Records whether feature f is missing	Flag indicating if a specific feature was originally missing
RMSE	Root mean squared error	Scalar metric	Regression evaluation	Typical error magnitude of the model's return predictions
MAE	Mean absolute error	Scalar metric	Regression evaluation	Average absolute magnitude of the return prediction errors
R^2	Coefficient of determination	Scalar metric	Variance explained	Proportion of return variance explained by the model
DA	Directional accuracy	Scalar metric	Sign agreement rate	Percentage of months the predicted return sign was correct
$\text{IC}(t)$	Monthly information coefficient	Scalar metric	Rank correlation within month	Correlation between predicted and actual stock rankings
$\text{TopK}(t)$	Top- K monthly realized return	Scalar metric	Portfolio-style metric	Average true return of stocks predicted to perform best
VR	Variance reduction	Scalar metric	Residual improvement diagnostic	The degree to which CNN reduced CatBoost's error variance